

Documentación y reporte: Proyecto final bases de datos

@Char[50] *Carlos T. Mendoza*

12 de julio de 2026

Índice

I. Introduction	2
a. Objetivos	2
II. Estructura general	3
a. Diagrama entidad relación (Workbench)	3
b. Diagrama con la 3ra Forma Normal (UML)	4
c. Justificación	4
III. Scripts SQL	5
a. Tablas e índices	5
b. Datos de prueba	7
c. Consultas y procedimientos	10
IV. Pruebas	16
V. Propuestas de Mejora	20
VI. Conclusión	21

I. Introduction

Se nos ha encomendado la tarea de crear una base de datos con la cual una pequeña tienda de productos electrónicos pueda gestionar todas sus operaciones. Las cuáles incluyen:

1. Gestionar los productos en *Stock*.
2. Gestionar la información de los clientes.
3. Manejar pedidos.
4. Permitir reseñas.
5. Organizar productos por categoría.

Además cuenta con restricciones fundamentales para evitar que se violen las reglas del negocio:

- Cada cliente puede realizar un máximo de 5 pedidos.
- El *Stock* no puede ser negativo.
- Solo se permiten reseñas de un cliente que haya comprado el producto.

a. Objetivos

El producto final de este proyecto debe cumplir los siguientes objetivos:

- Tener una base de datos relacional Normalizada(3FN)
- Implementar 3 consultas y 8 procedimientos almacenados.
- La base de datos debe estar optimizada mediante índices.
- Debe validarse la efectividad de la base de datos mediante pruebas.

II. Estructura general

a. Diagrama entidad relación (Workbench)

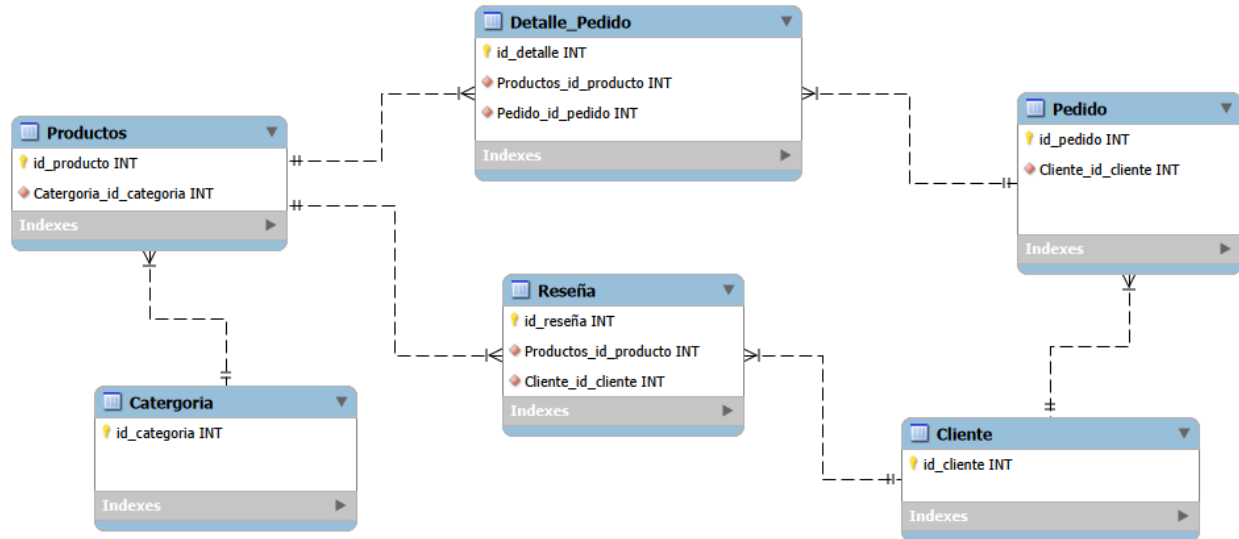


Figura 1: Primer boceto entidad-relación.

En este boceto podemos ver plasmadas las diversas entidades que nos van a permitir implementar cada una de las características fundamentales del negocio en cuestión, así como sus respectivas relaciones iniciales. Es importante mencionar que incluimos una tabla *Detalle_Pedido* para manejar de forma más eficiente la relación inherente entre el producto y su respectivo pedido.

b. Diagrama con la 3ra Forma Normal (UML)

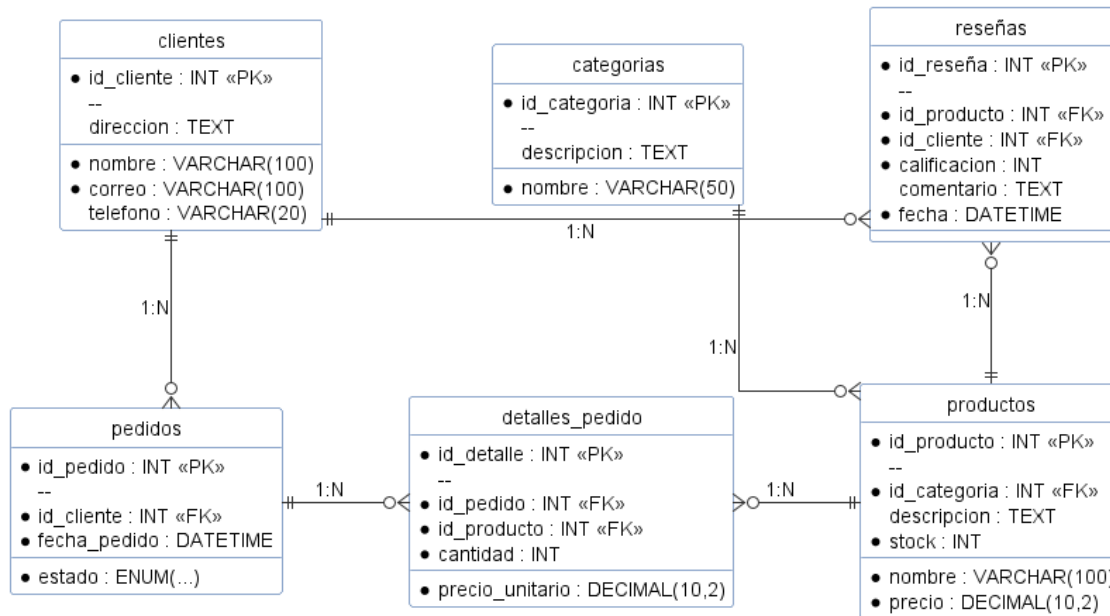


Figura 2: Diagrama UML Normalizado

En este diagrama se puede apreciar un modelo entidad relación mucho más completo y adecuado a la normalización seleccionada «3FN». La cuál cumple con los siguientes puntos:

1. Todas las tablas tienen una clave primaria atómica.
2. No hay dependencias transitivas.

c. Justificación

Puesto que la implementación solicitada involucraba evitar cualquier tipo de redundancia, era crucial lograr un esquema lógico que mitigara cualquier tipo de anomalía durante la inserción, borrado y actualización de valores. Por lo cual era necesario integrar la «**Primer Forma Normal**» en el desarrollo de nuestra estructura. Además cada tabla debía estar definida mediante campos atómicos, asegurando que cada columna tuviese un sólo valor correspondiente. Evitando por completo la multiplicidad de valores e integrando exitosamente la «**Segunda Forma Normal**». Para al final de todo, obtener tablas en las cuales no existe ningún tipo de dependencia parcial, lo que nos garantiza que cada columna que no es una «llave» dependa enteramente de la clave primaria de su respectiva tabla. Finalizando así por integrar la «**Tercera Forma Normal**».

III. Scripts SQL

a. Tablas e índices

```
CREATE DATABASE IF NOT EXISTS tienda_electronica;
USE tienda_electronica;
```

```
CREATE TABLE Categorias (
    id_categoria INT AUTO_INCREMENT,
    nombre VARCHAR(50) NOT NULL,
    descripcion TEXT,
    CONSTRAINT PK_Categorias PRIMARY KEY (id_categoria),
    CONSTRAINT UQ_Categoria_Nombre UNIQUE (nombre)
) ENGINE=InnoDB;
```

```
CREATE TABLE Clientes (
    id_cliente INT AUTO_INCREMENT,
    nombre VARCHAR(100) NOT NULL,
    correo VARCHAR(100) NOT NULL,
    telefono VARCHAR(20),
    direccion TEXT,
    CONSTRAINT PK_Clientes PRIMARY KEY (id_cliente),
    CONSTRAINT UQ_Cliente_Correo UNIQUE (correo)
) ENGINE=InnoDB;
```

```
CREATE TABLE Productos (
    id_producto INT AUTO_INCREMENT,
    id_categoria INT NOT NULL,
    nombre VARCHAR(100) NOT NULL,
    descripcion TEXT,
    precio DECIMAL(10,2) UNSIGNED NOT NULL,
    stock INT UNSIGNED NOT NULL DEFAULT 0,
    CONSTRAINT PK_Productos PRIMARY KEY (id_producto),
    CONSTRAINT FK_Productos_Categorias FOREIGN KEY (id_categoria)
        REFERENCES Categorias (id_categoria)
        ON DELETE RESTRICT ON UPDATE CASCADE
) ENGINE=InnoDB;
```

```
CREATE TABLE Pedidos (
    id_pedido INT AUTO_INCREMENT,
    id_cliente INT NOT NULL,
    fecha_pedido DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    estado ENUM('pendiente', 'enviado', 'entregado') NOT NULL DEFAULT 'pendiente',
```

```

        CONSTRAINT PK_Pedidos PRIMARY KEY (id_pedido),
        CONSTRAINT FK_Pedidos_Clientes FOREIGN KEY (id_cliente)
            REFERENCES Clientes (id_cliente)
            ON DELETE RESTRICT ON UPDATE CASCADE
    ) ENGINE=InnoDB;

CREATE TABLE Detalles_Pedido (
    id_detalle INT AUTO_INCREMENT,
    id_pedido INT NOT NULL,
    id_producto INT NOT NULL,
    cantidad INT UNSIGNED NOT NULL DEFAULT 1,
    precio_unitario DECIMAL(10,2) UNSIGNED NOT NULL,
    CONSTRAINT PK_Detalles_Pedido PRIMARY KEY (id_detalle),
    CONSTRAINT FK_Detalles_Pedidos FOREIGN KEY (id_pedido)
        REFERENCES Pedidos (id_pedido)
        ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT FK_Detalles_Productos FOREIGN KEY (id_producto)
        REFERENCES Productos (id_producto)
        ON DELETE RESTRICT ON UPDATE CASCADE
    ) ENGINE=InnoDB;

CREATE TABLE Reseñas (
    id_reseña INT AUTO_INCREMENT,
    id_producto INT NOT NULL,
    id_cliente INT NOT NULL,
    calificacion INT UNSIGNED NOT NULL,
    comentario TEXT,
    fecha DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT PK_Reseñas PRIMARY KEY (id_reseña),
    CONSTRAINT FK_Reseñas_Productos FOREIGN KEY (id_producto)
        REFERENCES Productos (id_producto)
        ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT FK_Reseñas_Clientes FOREIGN KEY (id_cliente)
        REFERENCES Clientes (id_cliente)
        ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT CK_Calificacion CHECK (calificacion BETWEEN 1 AND 5)
    ) ENGINE=InnoDB;

CREATE INDEX idx_productos_precio ON Productos(precio);
CREATE INDEX idx_pedidos_fecha_estado ON Pedidos(fecha_pedido, estado);
CREATE INDEX idx_reseñas_producto_fecha ON Reseñas(id_producto, fecha);

```

b. Datos de prueba

USE tienda_electronica;

INSERT INTO Categorías (nombre, descripción) VALUES

```
('Celulares Premium', 'Smartphones de gama alta y flagship de las principales marcas.'),
('Celulares Gama Media', 'Teléfonos inteligentes con excelente relación calidad-precio.'),
('Tablets', 'Dispositivos portátiles de pantalla grande para productividad y consumo multimedia.'),
('Audífonos True Wireless', 'Auriculares inalámbricos de botón (In-ear) con estuche de carga.'),
('Audífonos Over-Ear', 'Auriculares de diadema con cancelación activa de ruido (ANC).'),
('Relojes Inteligentes', 'Smartwatches enfocados en deporte, salud y notificaciones.'),
('Pulseras de Actividad', 'Smartbands accesibles para monitoreo físico diario.'),
('Cargadores y Energía', 'Adaptadores de carga rápida, cargadores inalámbricos y powerbanks.'),
('Fundas y Protección', 'Carcasas oficiales y protectores de pantalla para dispositivos.'),
('Laptops y Convertibles', 'Equipos portátiles premium y dispositivos 2 en 1.');
```

INSERT INTO Clientes (nombre, correo, telefono, direccion) VALUES

```
('Carlos Mendoza', 'carlos.mendoza@email.com', '5512345678', 'Av. Reforma 123, CDMX'),
('Ana Rodríguez', 'ana.rod@email.com', '5598765432', 'Calle Juárez 45, Guadalajara'),
('Sofía López', 'sofia.lopez@email.com', '8111223344', 'Col. Centro, Monterrey'),
('Sergio De Luis', 'sergio.deluis@email.com', '5544332211', 'Huixquilucan, Estado de México'),
('Andrea Sánchez', 'andrea.sanchez@email.com', '5566778899', 'El Pedregal, CDMX'),
('Luis Fernando Gómez', 'luis.gomez@email.com', '3344556677', 'Av. Vallarta 789, Guadalajara'),
('María Elena Cruz', 'maria.cruz@email.com', '2299887766', 'Boca del Río, Veracruz'),
('Alejandro Ruiz', 'alex.ruiz@email.com', '8188776655', 'San Pedro Garza García, NL'),
('Laura Flores', 'laura.flores@email.com', '5522446688', 'Coyoacán, CDMX'),
('Ricardo Morales', 'ricardo.m@email.com', '4422334455', 'Juriquilla, Querétaro'),
('Claudia Ortiz', 'claudia.o@email.com', '7771112233', 'Cuernavaca, Morelos'),
('Javier Herrera', 'javier.h@email.com', '9998887766', 'Mérida, Yucatán'),
('Gabriela Silva', 'gaby.silva@email.com', '6641112233', 'Tijuana, Baja California'),
('Daniel Castro', 'daniel.c@email.com', '4433221100', 'Morelia, Michoacán'),
('Patricia Méndez', 'paty.mendez@email.com', '2281112233', 'Pacho Nuevo, Veracruz'),
('Cliente Extra 16', 'extra16@email.com', '5511111111', 'Dirección 16'),
('Cliente Extra 17', 'extra17@email.com', '5522222222', 'Dirección 17'),
('Cliente Extra 18', 'extra18@email.com', '5533333333', 'Dirección 18'),
('Cliente Extra 19', 'extra19@email.com', '5544444444', 'Dirección 19'),
('Cliente Extra 20', 'extra20@email.com', '5555555555', 'Dirección 20');
```

INSERT INTO Productos (id_categoria, nombre, descripción, precio, stock) VALUES

```
(1, 'Samsung Galaxy S25 FE', 'Pantalla Dynamic AMOLED 2X, 128GB, Ecosistema Galaxy.', 13499.00, 23),
(1, 'Apple iPhone 15 Pro', 'Chasis de titanio, chip A17 Pro, 256GB de almacenamiento.', 23999.00, 10),
(1, 'Google Pixel 8 Pro', 'Cámara avanzada con IA de Google, 128GB, Android Puro.', 17499.00, 8),
(2, 'Xiaomi Redmi Note 13 Pro', 'Cámara de 200MP, carga rápida de 67W, 256GB.', 6499.00, 30),
(2, 'Motorola Edge 50 Neo', 'Pantalla pOLED, diseño estilizado, 256GB de almacenamiento.', 7999.00, 15).
```

```
(3, 'Apple iPad Air M2', 'Pantalla Liquid Retina de 11 pulgadas, 128GB, Wi-Fi.', 12999.00, 12),
(3, 'Huawei MatePad Pro 12.2', 'Pantalla Tandem OLED PaperMatte, ideal para diseño.', 19999.00, 7),
(4, 'OnePlus Buds Pro 3', 'Cancelación de ruido adaptativa dual, co-diseñados con Dynaudio.', 2999.00, 4),
(4, 'Oppo Enco X3', 'Audio de alta resolución, cancelación activa de ruido profunda.', 2499.00, 35),
(5, 'Sony WH-1000XM5', 'Audífonos premium de diadema con la mejor cancelación de ruido.', 6999.00, 20),
(6, 'Samsung Galaxy Watch 7', 'Monitoreo avanzado de salud, composition corporal y GPS dual.', 5499.00, 1),
(6, 'Honor Watch 4', 'Pantalla AMOLED de 1.75 pulgadas, batería de hasta 14 días.', 2199.00, 22),
(7, 'Xiaomi Smart Band 9', 'Pantalla AMOLED a 60Hz, monitoreo de ritmo cardíaco y sueño.', 899.00, 100),
(10, 'Lenovo ThinkPad X13 2-in-1', 'Intel Core Ultra 5, 32GB RAM, convertible ideal para desarrollo.', 2499.00, 1),
(10, 'Huawei MateBook X Pro', 'Ultra ligera, procesador Core Ultra 7, pantalla táctil 3.1K.', 31999.00, 1)
```

```
INSERT INTO Pedidos (id_pedido, id_cliente, fecha_pedido, estado) VALUES
```

```
(1, 1, '2026-07-01 10:00:00', 'entregado'),
(2, 2, '2026-07-02 11:15:00', 'entregado'),
(3, 3, '2026-07-03 14:30:00', 'enviado'),
(4, 4, '2026-07-04 09:00:00', 'entregado'),
(5, 5, '2026-07-04 16:45:00', 'entregado'),
(6, 6, '2026-07-05 12:20:00', 'pendiente'),
(7, 7, '2026-07-05 18:10:00', 'entregado'),
(8, 8, '2026-07-06 08:00:00', 'enviado'),
(9, 9, '2026-07-06 15:35:00', 'pendiente'),
(10, 10, '2026-07-07 11:40:00', 'entregado'),
(11, 11, '2026-07-07 19:50:00', 'entregado'),
(12, 12, '2026-07-08 10:05:00', 'enviado'),
(13, 13, '2026-07-08 14:15:00', 'pendiente'),
(14, 14, '2026-07-09 09:30:00', 'entregado'),
(15, 15, '2026-07-09 17:00:00', 'entregado'),
(16, 1, '2026-07-10 13:22:00', 'pendiente'),
(17, 4, '2026-07-10 15:40:00', 'entregado'),
(18, 7, '2026-07-11 08:12:00', 'pendiente'),
(19, 10, '2026-07-11 10:00:00', 'enviado'),
(20, 2, '2026-07-11 11:30:00', 'pendiente'),
(21, 1, '2026-07-11 10:19:33', 'pendiente');
```

```
INSERT INTO Detalles_Pedido (id_pedido, id_producto, cantidad, precio_unitario) VALUES
```

```
(1, 14, 1, 24999.00), (1, 1, 1, 13499.00),
(2, 6, 1, 12999.00),
(3, 2, 1, 23999.00), (3, 13, 2, 899.00),
(4, 11, 1, 5499.00), (4, 8, 1, 2999.00),
(5, 4, 1, 6499.00),
(6, 15, 1, 31999.00),
(7, 3, 1, 17499.00),
(8, 5, 1, 7999.00), (8, 12, 1, 2199.00),
(9, 9, 1, 2499.00),
```

```
(10, 13, 4, 899.00),
(11, 1, 1, 13499.00),
(12, 10, 1, 6999.00),
(13, 7, 1, 19999.00),
(14, 14, 1, 24999.00),
(15, 2, 1, 23999.00),
(16, 8, 1, 2999.00),
(17, 1, 1, 13499.00), (17, 11, 1, 5499.00),
(18, 3, 1, 17499.00),
(19, 6, 1, 12999.00),
(20, 12, 1, 2199.00),
(21, 1, 2, 13499.00);
```

```
INSERT INTO Reseñas (id_producto, id_cliente, calificacion, comentario, fecha) VALUES
```

```
(14, 1, 5, 'Excelente rendimiento en desarrollo. La pantalla táctil responde de maravilla.', '2026-07-03 12:00:00'),
(1, 1, 5, 'El Galaxy S25 FE va sumamente fluido, la cámara y el tamaño son perfectos.', '2026-07-03 12:00:00'),
(6, 2, 4, 'Excelente pantalla y muy ligera, el chip M2 responde de inmediato.', '2026-07-04 15:00:00'),
(2, 3, 5, 'El titanio se siente muy premium y las fotos de noche son espectaculares.', '2026-07-05 10:30:00'),
(11, 4, 5, 'Galaxy Watch 7 mide la composición corporal con precisión y el diseño es limpio.', '2026-07-06 09:15:00'),
(8, 4, 4, 'Muy buena cancelación de ruido, los graves se escuchan potentes.', '2026-07-06 09:15:00'),
(4, 5, 4, 'Por el precio que tiene, la cámara de 200MP y la carga rápida son imbatibles.', '2026-07-06 11:00:00'),
(3, 7, 5, 'Fotografía computacional pura. El software de Google no decepciona.', '2026-07-07 14:22:00'),
(5, 8, 4, 'El rendimiento del Edge 50 es muy balanceado y la pantalla pOLED destaca.', '2026-07-08 16:40:00'),
(12, 8, 4, 'Sencillo, cómodo y la batería dura semanas. Gran reloj de Honor.', '2026-07-08 16:45:00'),
(13, 10, 5, 'Increíble relación calidad-precio. La pantalla a 60Hz se nota muy suave.', '2026-07-09 13:00:00'),
(14, 14, 5, 'El teclado de la ThinkPad sigue siendo el mejor para escribir código.', '2026-07-10 11:10:00'),
(2, 15, 5, 'Cámara perfecta y excelente rendimiento con juegos pesados.', '2026-07-10 18:00:00'),
(1, 4, 5, 'Gran integración con el ecosistema de pantallas y la suite de Samsung.', '2026-07-11 09:00:00'),
(10, 12, 5, 'Cancelación de sonido de primer nivel, ideales para aislarse en la oficina.', '2026-07-09 11:00:00'),
(7, 13, 5, 'La pantalla Tandem OLED de Huawei es una locura de brillo y colores.', '2026-07-10 10:00:00'),
(9, 9, 4, 'Audio nítido y se conectan instantáneamente con mi teléfono Oppo.', '2026-07-10 10:15:00'),
(13, 11, 4, 'Cómoda y ligera, perfecta para llevar el registro de pasos diario.', '2026-07-09 11:00:00'),
(1, 17, 5, 'Una opción muy sólida y balanceada de Samsung.', '2026-07-11 11:00:00'),
(6, 19, 5, 'El complemento perfecto si ya usas una computadora Apple.', '2026-07-11 11:15:00'),
(1, 1, 5, 'Componentes premium y gran desempeño en Ubuntu.', '2026-07-11 10:20:43');
```

c. Consultas y procedimientos

```
-- //Vistas// --
```

```
-- 1. Listar productos disponibles por categoría, ordenados por precio.
```

```
CREATE OR REPLACE VIEW Vista_Productos_Disponibles AS
```

```
SELECT
```

```
    c.nombre Categoria,  
    p.nombre Producto,  
    p.precio Precio,  
    p.stock Unidades_Disponibles
```

```
FROM Productos p
```

```
JOIN Categorías c ON p.id_categoria = c.id_categoria --> Se realiza un JOIN (INNER JOIN) para encontrar
```

```
WHERE p.stock > 0 --> Filtramos para obtener solo productos con almenos 1 pieza en Stock.
```

```
ORDER BY c.nombre ASC, p.precio ASC; --> Ordenamos en sentido ascendente por el nombre del cliente y el p
```

```
-- 2. Mostrar clientes con pedidos pendientes y total de compras.
```

```
CREATE OR REPLACE VIEW Vista_Clientes_Pedidos_Pendientes AS
```

```
SELECT
```

```
    c.id_cliente,  
    c.nombre Cliente,
```

```
--> Solo suma cuando el estado del pedido se encuentra 'pendiente'.
```

```
SUM(CASE WHEN p.estado = 'pendiente' THEN 1 ELSE 0 END) AS Pedidos_Pendientes,
```

```
--> Comprobación para valores nulos, devuelve 0 si la suma de los productos (cantidad * precio_unitario)
```

```
IFNULL(SUM(dp.cantidad * dp.precio_unitario), 0) AS Total_Historico_Compras
```

```
FROM Clientes c
```

```
JOIN Pedidos p ON c.id_cliente = p.id_cliente
```

```
JOIN Detalles_Pedido dp ON p.id_pedido = dp.id_pedido
```

```
GROUP BY c.id_cliente, c.nombre --> Agrupamos según el ID de cliente y su nombre
```

```
HAVING Pedidos_Pendientes > 0; --> Filtramos solo aquellos que tienen almenos un pedido pendiente.
```

```
-- 3. Reporte de los 5 productos con mejor calificación promedio en reseñas.
```

```
CREATE OR REPLACE VIEW Vista_Top5_Productos_Resenas AS
```

```
SELECT
```

```
    p.id_producto,
```

```
    p.nombre AS Producto,
```

```
--> Redondeamos el promedio de la calificación a dos decimales.
```

```
ROUND(AVG(r.calificacion), 2) AS Calificacion_Promedio,
```

```
--> Contamos el total de reseñas.
```

```
COUNT(r.id_reseña) AS Total_Reseñas
```

```
FROM Productos p
```

```
JOIN Reseñas r ON p.id_producto = r.id_producto
```

```
GROUP BY p.id_producto, p.nombre --> Agrupados por ID y nombre del producto
```

```
ORDER BY Calificacion_Promedio DESC, Total_Reseñas DESC --> Ordenados por el promedio de calificación y
LIMIT 5; --> Marcamos un límite de 5 productos a mostrar.
```

```
-- //Procedimientos// --
```

```
-- 1. Registrar un nuevo pedido (verificando límite de 5 pendientes y stock).
```

```
DELIMITER //
```

```
DROP PROCEDURE IF EXISTS `SP_RegistrarPedido`// --> Comprobación general para evitar errores de creación
```

```
CREATE PROCEDURE `SP_RegistrarPedido` (
```

```
    IN p_id_cliente INT,
```

```
    IN p_id_producto INT,
```

```
    IN p_cantidad INT
```

```
)
```

```
BEGIN
```

```
    DECLARE v_pedidos_pendientes INT;
```

```
    DECLARE v_stock_actual INT;
```

```
    DECLARE v_precio_unitario DECIMAL(10,2);
```

```
    DECLARE v_id_pedido INT;
```

```
--> Verifica la cantidad de pedidos pendientes del cliente.
```

```
SELECT COUNT(*) INTO v_pedidos_pendientes
```

```
FROM Pedidos
```

```
WHERE id_cliente = p_id_cliente AND estado = 'pendiente';
```

```
IF v_pedidos_pendientes >= 5 THEN
```

```
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Error: El cliente ya tiene el máximo de 5 pedidos p
```

```
END IF;
```

```
--> Verifica el stock y obtiene el precio del producto.
```

```
SELECT stock, precio INTO v_stock_actual, v_precio_unitario
```

```
FROM Productos WHERE id_producto = p_id_producto;
```

```
IF v_stock_actual < p_cantidad THEN
```

```
    --> Mensaje de error y señal de paro total para el sistema.
```

```
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Error: Stock insuficiente para el producto solicitad
```

```
END IF;
```

```
--> Inserción del pedido
```

```
INSERT INTO Pedidos (id_cliente, fecha_pedido, estado) VALUES (p_id_cliente, NOW(), 'pendiente');
```

```
SET v_id_pedido = LAST_INSERT_ID();
```

```

--> Inserción del detalle
INSERT INTO Detalles_Pedido (id_pedido, id_producto, cantidad, precio_unitario)
VALUES (v_id_pedido, p_id_producto, p_cantidad, v_precio_unitario);

--> Se actualiza el stock mandando a llamar la Routine correspondiente.
CALL SP_ActualizarStock(p_id_producto, p_cantidad);
END //
DELIMITER ;

-- 2. Registrar una reseña.
DELIMITER //
DROP PROCEDURE IF EXISTS `SP_RegistrarReseña`//
CREATE PROCEDURE `SP_RegistrarReseña`(
    IN p_id_producto INT,
    IN p_id_cliente INT,
    IN p_calificacion INT,
    IN p_comentario TEXT
)
BEGIN
    INSERT INTO Reseñas (id_producto, id_cliente, calificacion, comentario, fecha)
    -- El Trigger 'Before_Insert_Reseña' validará automáticamente si el cliente compró el producto.
    -- Si no lo compró, el trigger abortará el INSERT automáticamente lanzando un error 45000.
    VALUES (p_id_producto, p_id_cliente, p_calificacion, p_comentario, NOW());
END //
DELIMITER ;

-- 3. Actualizar el stock de un producto después de un pedido.
DELIMITER //
DROP PROCEDURE IF EXISTS `SP_ActualizarStock`//
CREATE PROCEDURE `SP_ActualizarStock`(
    IN p_id_producto INT,
    IN p_cantidad_restar INT
)
BEGIN
    UPDATE Productos
    --> El Stock se reduce X cantidad según el pedido del producto realizado.
    SET stock = stock - p_cantidad_restar
    WHERE id_producto = p_id_producto;
END //
DELIMITER ;

```

```

-- 4. Cambiar el estado de un pedido.
DELIMITER //
DROP PROCEDURE IF EXISTS `SP_CambiarEstadoPedido`//
CREATE PROCEDURE `SP_CambiarEstadoPedido`(
    IN p_id_pedido INT,
    IN p_nuevo_estado ENUM('pendiente', 'enviado', 'entregado')
)
BEGIN
    UPDATE Pedidos
    --> Definimos el estado según sea el caso.
    SET estado = p_nuevo_estado
    WHERE id_pedido = p_id_pedido;
END //
DELIMITER ;

-- 5. Eliminar reseñas de un producto específico.
DELIMITER //
DROP PROCEDURE IF EXISTS `SP_EliminarReseñasProducto`//
CREATE PROCEDURE `SP_EliminarReseñasProducto`(
    IN p_id_producto INT
)
BEGIN
    DELETE FROM Reseñas WHERE id_producto = p_id_producto;
    SELECT 'Reseñas eliminadas. El promedio se actualizará en las consultas dinámicas.' AS Resultado;
END //
DELIMITER ;

-- 6. Agregar un nuevo producto verificando duplicados.
DELIMITER //
DROP PROCEDURE IF EXISTS `SP_AgregarProducto`//
CREATE PROCEDURE `SP_AgregarProducto`(
    IN p_id_categoria INT,
    IN p_nombre VARCHAR(100),
    IN p_descripcion TEXT,
    IN p_precio DECIMAL(10,2),
    IN p_stock INT
)
BEGIN
    DECLARE v_existe INT;

    --> Se realiza un conteo de productos para posteriormente validar si se intenta agregar un producto
    SELECT COUNT(*) INTO v_existe

```

```

FROM Productos
WHERE nombre = p_nombre AND id_categoria = p_id_categoria;

--> Señal de error y paro total de la Routine si el producto existe.
IF v_existe > 0 THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Error: Ya existe un producto con ese nombre en esta
END IF;

--> Si no es el caso se realiza la inserción del producto.
INSERT INTO Productos (id_categoria, nombre, descripcion, precio, stock)
VALUES (p_id_categoria, p_nombre, p_descripcion, p_precio, p_stock);
END //
DELIMITER ;

--7. Actualizar la información de un cliente.
DELIMITER //
DROP PROCEDURE IF EXISTS `SP_ActualizarCliente`//
CREATE PROCEDURE `SP_ActualizarCliente`(
    IN p_id_cliente INT,
    IN p_nueva_direccion TEXT,
    IN p_nuevo_telefono VARCHAR(20)
)
BEGIN
    UPDATE Clientes
    --> Actualizamos el nuevo valor de los atributos del cliente.
    SET direccion = p_nueva_direccion, telefono = p_nuevo_telefono
    WHERE id_cliente = p_id_cliente;
END //
DELIMITER ;

-- 8. Generar un reporte de productos con stock bajo (< 5 unidades).
DELIMITER //
DROP PROCEDURE IF EXISTS `SP_ReporteStockBajo`//
CREATE PROCEDURE `SP_ReporteStockBajo`()
BEGIN
    --> Tomamos toda la información relevante del producto.
    SELECT
        id_producto,
        nombre AS Producto,
        stock AS Unidades_Disponibles,
        precio
    FROM Productos

```

```
--> Filtramos aquellos que tengan menos de 5 piezas en Stock.  
WHERE stock < 5  
ORDER BY stock ASC;  
END //  
DELIMITER ;
```

IV. Pruebas

Top 5 Productos con Mejores Reseñas

Mediante la ejecución de esta consulta se verifica que el sistema calcula correctamente los promedios en tiempo real basándose en las calificaciones atómicas de los clientes.

```
1 SELECT * FROM vista_top5_productos_resenas;  
2
```

	id_producto	Producto	Calificacion_Promedio	Total_Reseñas
▶	1	Samsung Galaxy S25 FE	5.00	4
	2	Apple iPhone 15 Pro	5.00	2
	14	Lenovo ThinkPad X13 2-in-1	5.00	2
	3	Google Pixel 8 Pro	5.00	1
	10	Sony WH-1000XM5	5.00	1

Figura 3: Resultado prueba A.

Cientes con Pedidos Pendientes

Aqui podemos validar que el sistema este excluyendo adecuadamente los pedidos dependiendo de su estado actual (“enviado” o “entregado”).

```
1 SELECT * FROM vista_plientes_pedidos_pendientes LIMIT 3;  
2
```

	id_cliente	Cliente	Pedidos_Pendientes	Total_Historico_Compras
▶	1	Carlos Mendoza	2	68495.00
	2	Ana Rodríguez	1	15198.00
	6	Luis Fernando Gómez	1	31999.00

Figura 4: Resultado prueba C.

Productos Disponibles

Aquí podemos validar que el sistema este excluyendo adecuadamente los pedidos dependiendo de su estado actual (“enviado” o “entregado”).

```
1 SELECT * FROM vista_productos_disponibles;  
2
```

	Categoria	Producto	Precio	Unidades_Disponibles
►	Audifonos Over-Ear	Sony WH-1000XM5	6999.00	20
	Audifonos True Wireless	Oppo Enco X3	2499.00	35
	Audifonos True Wireless	OnePlus Buds Pro 3	2999.00	40
	Celulares Gama Media	Xiaomi Redmi Note 13 Pro	6499.00	30
	Celulares Gama Media	Motorola Edge 50 Neo	7999.00	15
	Celulares Premium	Samsung Galaxy S25 FE	13499.00	23
	Celulares Premium	Google Pixel 8 Pro	17499.00	8
	Celulares Premium	Apple iPhone 15 Pro	23999.00	10
	Laptops y Convertibles	Lenovo ThinkPad X13 2-in-1	24999.00	14
	Laptops y Convertibles	Huawei MateBook X Pro	31999.00	5
	Pulseras de Actividad	Xiaomi Smart Band 9	899.00	100
	Relojes Inteligentes	Honor Watch 4	2199.00	22
	Relojes Inteligentes	Samsung Galaxy Watch 7	5499.00	18
	Tablets	Apple iPad Air M2	12999.00	12
	Tablets	Huawei MatePad Pro 12.2	19999.00	7

Figura 5: Resultado prueba B.

Explicación de los índices creados y el impacto que tienen

Para analizar el rendimiento, ejecutamos una consulta de prueba incluyendo el comando `EXPLAIN` para ilustrar el impacto positivo que tiene cada índice y porqué.

■ `idx_productos_precio:`

```
1      EXPLAIN SELECT id_producto, nombre, precio
2      FROM Productos
3      WHERE precio
4      BETWEEN 5000.00 AND 15000.00;
```

Al momento de ejecutar dicha sentencia, el plan de ejecución muestra en la columna *type* el valor *range* y en la columna *key* confirma el uso del propio índice. Lo cuál significa que **MySQL** salta directamente a los bloques de memoria que contienen ese rango de precios, evitando por completo un *Full Table Scan*. Agilizando el filtrado durante tareas de búsqueda por rangos de presupuesto.

■ `idx_pedidos_fecha_estado:`

```
1      EXPLAIN SELECT id_pedido, id_cliente, fecha_pedido, estado
2      FROM Pedidos
3      WHERE fecha_pedido >= '2026-07-01 00:00:00'
4      AND fecha_pedido <= '2026-07-31 23:59:59'
5      AND estado = 'entregado';
```

Al ser un índice compuesto, la estructura del árbol *B-tree* del índice organiza primero por fecha y luego por estado. El **EXPLAIN** nos muestra en la columna *rows* tan solo una fracción del total de la tabla, puesto que el motor descarta cualquier registro fuera del mes especificado y que no cumpla con la condición de estado. Todo esto reduce drásticamente el consumo de *CPU*.

■ `idx_reseñas_producto_fecha:`

```
1      EXPLAIN SELECT id\_resena, calificacion, comentario, fecha
2      FROM Resenas
3      WHERE id_producto = 1
4      ORDER BY fecha DESC;
```

Esta es una de las optimizaciones más importantes. Sin este índice compuesto, al momento de utilizar un **ORDER BY**, la columna *Extra* del **EXPLAIN** arrojaría un costoso *Using Filesort* «Lo que significa que MySQL carga datos en la RAM para ordenarlos al vuelo». Gracias al índice, los registros del producto ya se guardan físicamente ordenados por fecha en el almacenamiento principal, por lo que el *filesort* desaparece por completo. Permitiendo que la vista de reseñas en el frontend cargue instantáneamente.

Pruebas de procedimientos almacenados

Se diseñaron casos de prueba para validar que las reglas de negocio dispuestas mediante los *Triggers* y *Procedures*, los cuales protegen la integridad de la base de datos.

Escenario A: Intento de reseña sin haber comprado el producto

Acción: El Cliente 3 intenta dejar una reseña para el Producto 14, el cual no ha comprado.

Comando: CALL SP_RegistrarReseña(14, 3, 5, 'Buen equipo');

Resultado: El sistema bloquea la transacción. El Trigger detona el error capturado:

Error Code: 1644. Error de integridad: No puedes dejar una reseña de un producto que no has comprado.

Figura 6: Resultado A.

Escenario B: Pedido exitoso con actualización de inventario

Acción: El Cliente 1 compra 2 unidades del Producto 13 (Stock inicial: 100).

Comando: CALL SP_RegistrarPedido(1, 13, 2);

Resultado: El pedido se registra como “pendiente”. Al consultar el inventario, este se reduce automáticamente a 98 mediante la subrutina de actualización de stock.

	22	1	2026-07-12 16:49:28	pendiente
--	----	---	---------------------	-----------

Figura 7: Resultado B1.

	id_producto	id_categoria	nombre	descripcion	precio	stock
►	13	7	Xiaomi Smart Band 9	Pantalla AMOLED a 60Hz, monitoreo de ritmo ca...	899.00	98

Figura 8: Resultado B2.

Escenario C: Bloqueo por falta de stock

Acción: Un cliente intenta comprar 30 unidades del Producto 2 (Stock actual: 10).

Comando: CALL SP_RegistrarPedido(2, 2, 30);

Resultado: La transacción falla de forma controlada lanzando el mensaje:

Error Code: 1644. Error: Stock insuficiente para el producto solicitado.

Figura 9: Resultado C.

V. Propuestas de Mejora

Tras analizar profundamente el esquema actual, consideramos que podrían implementarse las siguientes optimizaciones de cara a un entorno de producción más amplio:

1. Nuevo Índice en `Productos.stock`:

Algunas consultas, como la vista de productos disponibles, filtran constantemente mediante `WHERE stock > 0`. Por lo que agregar un índice `INDEX idx_stock (stock)` aceleraría la carga del catálogo en el frontend.

2. Particionamiento de la tabla `Pedidos`:

Si pensamos a largo plazo, mantener el historial de años de compras en una sola tabla degradaría cada vez más las búsquedas. Por lo que sería conveniente implementar *Table Partitioning* por año en la columna `fecha_pedido` para aislar los datos históricos de los pedidos activos.

3. Índice de Texto Completo (Full-Text Search):

Realizar la implementación de un índice `FULLTEXT(nombre, descripcion)` dentro de la tabla `Productos` podría optimizar la barra de búsqueda de la tienda, permitiendo al usuario buscar por palabras clave «como: “Laptop Intel”» de forma nativa sin usar costosos `LIKE '%termino%'`.

VI. Conclusión

A lo largo de este trabajo hemos podido poner en práctica todo nuestro conocimiento respecto a bases de datos y sistemas, para lograr construir un proyecto coherente y efectivo en función de los requisitos proporcionados. Además de los conocimientos previos, hemos adquirido nuevos gracias a la investigación referente a la optimización de la gestión de bases de datos. Con todo esto, nos queda bastante claro que todos y cada uno de los pequeños detalles que conforman este tipo de proyectos, tienen un peso enorme para la eficiencia y el correcto funcionamiento a nivel micro y macro de todo un sistema.

Sin lugar a duda, construir con detenimiento exhaustivo y entender el porqué detrás de cada ínfima implementación, se convierte en un peso más sobre la balanza que se inclina hacia el éxito de cualquier proyecto. En cuestiones como esta no sólo determina la efectividad de las consultas o la integridad de los datos, asegura paso a paso que el sistema que se cree al rededor de la base de datos tenga los mejores cimientos posibles. Lo que más adelante constituirá un motivo más por el cuál el mercado debería preferir nuestro sistema por encima del resto.

En proyectos como éste y otros tantos, seguimos reconociendo el valor tan gigantesco del ingenio y la originalidad por encima de un entendimiento superficial y un pensamiento cuadrado. Con ansias de muchos más retos que pongan a prueba nuestras habilidades y conocimiento, esto es todo... Por ahora.